



**BENCHMARK EMPÍRICO DAS LEIS DE AMDAHL E GUSTAFSON EM CPU SERIAL,
OPENMP E GPU CUDA**

**EMPIRICAL BENCHMARK OF AMDAHL'S AND GUSTAFSON'S LAWS ON SERIAL CPU,
OPENMP AND CUDA GPU**

Marcos Paulo Araujo Pereira, Vinicius Almeida Fajardo
Alex Vitorino

Rodovia MG 482 - Gigante, Conselheiro Lafaiete - MG, 36400-000 – Brasil
<241-006308@aluno.unipac.br>, <241-006741@aluno.unipac.br>

Resumo

O fim do escalonamento do clock, por limitações físicas, forçou a indústria a mudar de escala: em vez de processadores mais rápidos, passou a adicionar mais processadores em um único chip. Com isso, o ganho de desempenho passou a depender da relação entre software e hardware: o software precisa ser otimizado para a arquitetura em que executa, usando os recursos de forma inteligente, como o paralelismo, e o hardware precisa oferecer arquiteturas distintas para cenários de complexidade diferente. Este artigo é um experimento prático para investigar o paralelismo e suas limitações, explorando também como a complexidade do problema influencia o sistema de forma global. Os principais resultados mostram um ganho de speedup fortemente linear quando o problema é de cálculo denso. Observou-se ainda uma anomalia: enquanto em duas arquiteturas o tempo de transferência PCIe ficou entre 6% e 9%, em uma arquitetura com mais recursos computacionais ele subiu para 39%: o cálculo denso foi computado tão rápido que a transferência PCIe passou a pesar mais do que nas outras arquiteturas, demonstrando como o dimensionamento entre software e hardware é decisivo.

Palavras-chave: Lei de Amdahl. Lei de Gustafson. Paralelismo. Speedup. Intensidade operacional.

Abstract

The end of clock scaling, driven by physical limits, forced the industry to change scale: instead of faster processors, it began adding more processors on a single chip. Performance gains then came to depend on the relationship between software and hardware: software must be optimized for the architecture on which it runs, using resources intelligently, such as parallelism, while hardware must offer distinct architectures for problems of differing complexity. This article is a practical experiment investigating parallelism and its limitations, also exploring how a problem's complexity influences the system as a whole. The main results show a strongly linear speedup gain when the problem is compute-dense. An anomaly was also observed: while in two architectures the PCIe transfer time stayed between 6% and 9%, in an architecture with more computational resources it rose to 39%: the dense computation ran so fast that the PCIe transfer began to weigh more than in the other architectures, demonstrating how the sizing between software and hardware is decisive.

Keywords: Amdahl's law. Gustafson's law. Parallelism. Speedup. Operational intensity.

LISTA DE SIGLAS

BCE - *Base Core Equivalent* (equivalente de núcleo base)
CPU - *Central Processing Unit*
CUDA - *Compute Unified Device Architecture*
D2H - *Device to Host* (transferência VRAM para DRAM)
DRAM - *Dynamic Random Access Memory*
E/S - *Entrada/Saída (Input/Output)*
FLOP - *Floating-point Operation*
GPU - *Graphics Processing Unit*
H2D - *Host to Device* (transferência DRAM para VRAM)
LCG - *Linear Congruential Generator*
MPI - *Message Passing Interface*
OI - *Intensidade Operacional (Operational Intensity)*
OpenMP - *Open Multi-Processing*
PCIe - *Peripheral Component Interconnect Express*
RAM - *Random Access Memory*
SIMD - *Single Instruction, Multiple Data*
VRAM - *Video Random Access Memory*

1 INTRODUÇÃO

Durante décadas, o ganho de desempenho era diretamente ligado ao aumento de clock, pois a arquitetura era baseada em single-core: bastava aumentá-lo junto com a quantidade de transistores para ganhar desempenho a cada nova geração. O número de transistores dobrava a cada cerca de dois anos, conforme a Lei de Moore (MOORE, 1965). Esse aprimoramento constante fez a indústria chegar à limitação física causada pela potência exigida pelo alto clock, além da limitação de dissipação dessa potência. Com isso, a indústria foi obrigada a pausar o aumento de clock e a contornar o problema explorando outras arquiteturas: pensou-se em adicionar mais processadores em um único chip, ou seja, em vez de possuir um processador veloz, passou-se a possuir muitos processadores trabalhando em paralelo, além do uso de GPUs para computação de propósito geral. Nesse cenário, o desempenho não depende somente do hardware; introduziu-se a necessidade de o software saber trabalhar em paralelismo, utilizando de forma eficiente os N núcleos. Isso torna a entrega da solução uma atividade de engenharia, pois depende de quão complexo o problema é em relação aos recursos disponíveis, ou então de quantificar os recursos necessários para resolvê-lo, além de exigir lidar com várias arquiteturas diferentes. Muitos modelos clássicos colaboram no entendimento dos sistemas computacionais: a Lei de Amdahl (1967) e a Lei de Gustafson (1988), mesmo formuladas para sistemas diferentes dos atuais; Hill & Marty (2008), com a exploração de arquiteturas diferentes de mesmo custo; e o Modelo Roofline (Williams, Waterman & Patterson, 2009), que colabora para o entendimento

de onde mora o gargalo dos sistemas computacionais. Este trabalho demonstra de forma básica as propriedades de um sistema computacional, explorando métricas teóricas com as leis de Amdahl e Gustafson e comparando-as com a experimentação feita em diferentes modos de processamento CPU serial, CPU paralela (OpenMP) e GPU (CUDA) utilizando algoritmos de busca em estrutura de dados e algoritmos onde as operações de cálculo são densas.

1.1 Problema de pesquisa

Em que medida as leis de Amdahl e Gustafson são capazes de explicar, na prática, o ganho de desempenho (speedup) obtido por algoritmos distintos com mesmo objetivo, executados em CPU serial, CPU paralela (OpenMP) e GPU (CUDA), e como o volume de dados influencia esse comportamento?

1.2 Justificativa

Este artigo contribui para o desenvolvimento do senso crítico, demonstrando a limitação do paralelismo. É comum a intuição associar “mais núcleos” ou “mais paralelismo em um software” a um ganho garantido de desempenho. Este artigo demonstra que, por meio de experimentação científica, é possível encontrar qual é a arquitetura ideal e como o software se beneficia nessa arquitetura. É possível distinguir a diferença entre CPU e GPU e mostrar a principal limitação de potencial da GPU, e como desperdiçar transístores fazendo núcleos complexos colabora em atividades limitadoras, como a fração serial, colaborando com todo o sistema o que mostra a falha do senso comum de que desperdiçar recursos é ruim. Talvez o principal ponto seja como um erro de arquitetura pode custar caro em um sistema real, com milhares ou bilhões de operações.

1.3 Objetivos

Objetivo geral

- Observar as limitações do paralelismo em cenários diferentes (CPU serial, OpenMP e GPU CUDA), usando como métrica as leis de Amdahl e Gustafson e os limites dessas equações.

Objetivos específicos

- Implementar os algoritmos nas 3 variantes (serial, OpenMP e CUDA) e avaliar os resultados.
- Medir tempo de execução, speedup e eficiência por configuração e por volume de dados N .

- Confrontar os valores medidos com as previsões de Amdahl e Gustafson, identificando onde as leis divergem do observado.
- Explorar a influência de cargas memory-bound e compute-bound no ganho de speedup.

2 REFERENCIAL TEÓRICO

Esta seção aborda os principais conceitos utilizados nesse artigo, Box-Muller utilizado para geração de dados fictícios próximos à realidade, esses dados foram utilizados nos algoritmos de busca, Leis de Amdahl, Gustafson foi utilizado suas equações para métricas e visões, Modelo Roofline utilizado seus conceitos de intensidade operacional como meio de justificar a limitação de ganho de speedup, Hill & Marty (2008) trouxe uma visão da complexidade de núcleos e suas arquiteturas colaborando de forma geral no entendimento da distinção entre o propósito da CPU e GPU e também abertura para trabalhos futuros.

2.1 Box-Muller

A técnica de Box-Muller (BOX; MULLER, 1958) utiliza conjunto de números gerados aleatoriamente U_1 e U_2 no intervalo de 0 e 1, são gerados pela função `rand()`, essa técnica consiste em utilizar o plano cartesiano onde se faz o uso do raio que parte do ponto 0,0 calculado pela fórmula $\sqrt{-2 \cdot \ln(U_1)}$ observe que U_1 é um número uniforme aleatório, após isso é utilizado o cálculo $2 \times \pi \times U_2$ para gerar um número em radianos que representa o ângulo (θ) para onde o raio aponta; depois é utilizado $\text{Raio} \times \cos(\theta)$ e também SEN no lugar do $\cos$, que entrega o número aleatório distribuído de forma que gere um gráfico de sino, observe que são duas entradas e duas saídas, as figuras abaixo demonstram a distribuição normal e uniforme:

As duas saídas formam o par canônico, geradas a partir do mesmo par (U_1, U_2):

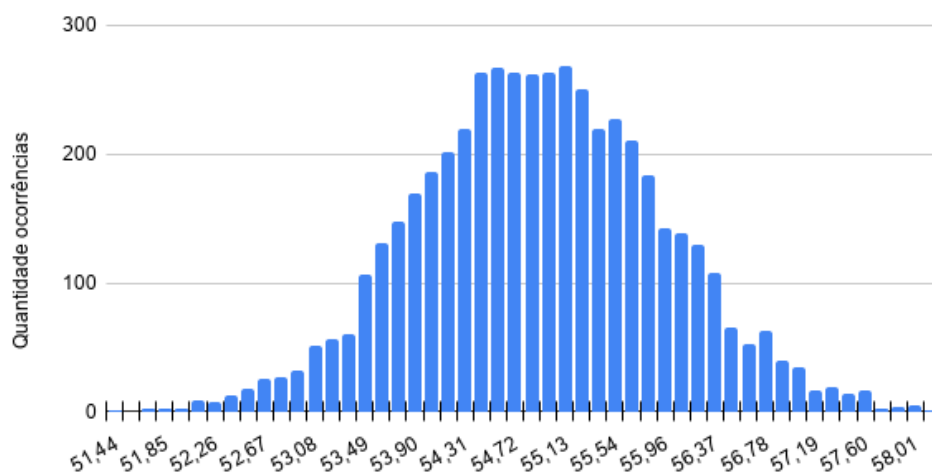
$$Z_0 = \sqrt{-2 \ln U_1} \cos(2\pi U_2) \quad Z_1 = \sqrt{-2 \ln U_1} \sin(2\pi U_2) \quad (1)$$

Repare que o custo de `log` e `sqrt` é pago uma vez e amortizado entre os dois números: cada par de uniformes produz **dois** valores normais.

Figura 1 – Distribuição normal utilizando método Box-Muller onde foram gerados 10.000 números uniforme (U1 e U2) no intervalo entre 0 e 1, é observado e esperado que a distribuição em seu centro possua alta probabilidade e ocupação de números gerados e a partir de seu centro seus lados sejam espelhados como gráfico de sino

Técnica Box-Muller

10.000 Números distribuição normal, média 55

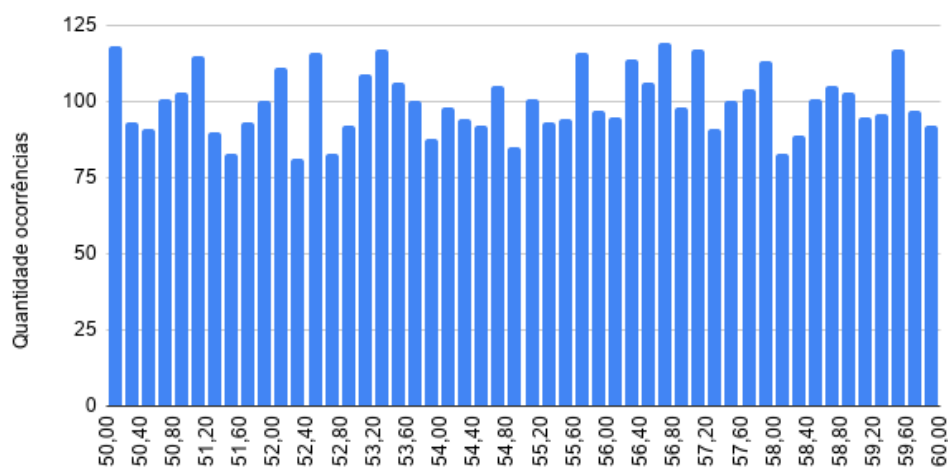


Fonte: Elaborada pelo autor.

Figura 2 – Distribuição uniforme onde todos os números gerados possuem a mesma probabilidade de serem gerados

Números uniformes gerados aleatoriamente

10.000 Números gerados usando função rand()



Fonte: Elaborada pelo autor.

No benchmark foi utilizado a técnica de Box-Muller prezando a qualidade realística de dados fictícios onde representa leitura de sensores genéricos.

2.2 Leis de Amdahl e Gustafson

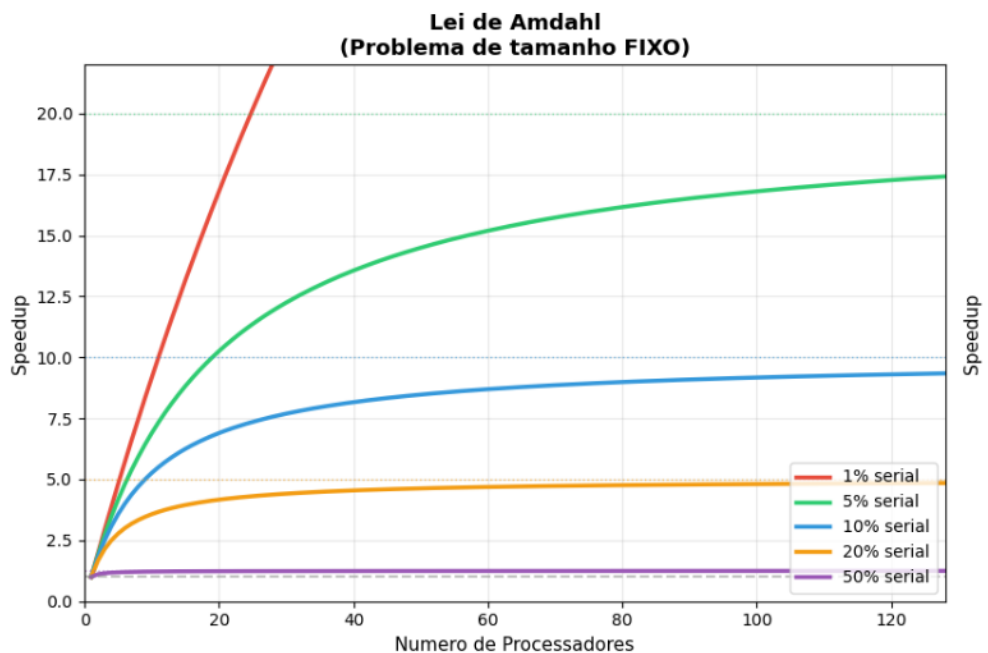
Lei de Amdahl: A Perspectiva de Carga Fixa Criada por Gene Amdahl em 1967, a Lei de Amdahl assume que o tamanho do problema é fixo, com um simples objetivo de responder a questão de quanto tempo podemos economizar ao executar uma tarefa de tamanho fixo utilizando mais processadores? Qualquer programa possui uma fração de código que pode ser executada de forma estritamente paralela (P) e uma porção que é necessariamente sequencial ou serial (1 - P), como rotinas de inicialização, operações de entrada/saída (E/S) e sincronização de threads (AMDAHL, 1967).

Fórmula do Speedup de Amdahl O ganho de velocidade teórico (speedup) obtido ao distribuir o código por N processadores é expresso por:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}} \quad (2)$$

P = Quantidade do código em paralelo; N = Quantidade de processadores.

Figura 3 – Curva teórica da Lei de Amdahl speedup S(N) em função do número de processadores N para diferentes valores da fração paralela P (50%, 75%, 90%, 95%). Demonstra a saturação do ganho quando a fração serial domina, limitando o speedup máximo a 1/(1-P)



Fonte: Elaborada pelo autor (Google Colab).

Fórmula de Eficiência de Amdahl A eficiência (E) mede o nível de aproveitamento dos processadores disponíveis, sendo calculada a partir da divisão do speedup obtido pelo número de processadores N:

$$E(N) = \frac{S(N)}{N} = \frac{1}{N(1 - P) + P} \quad (3)$$

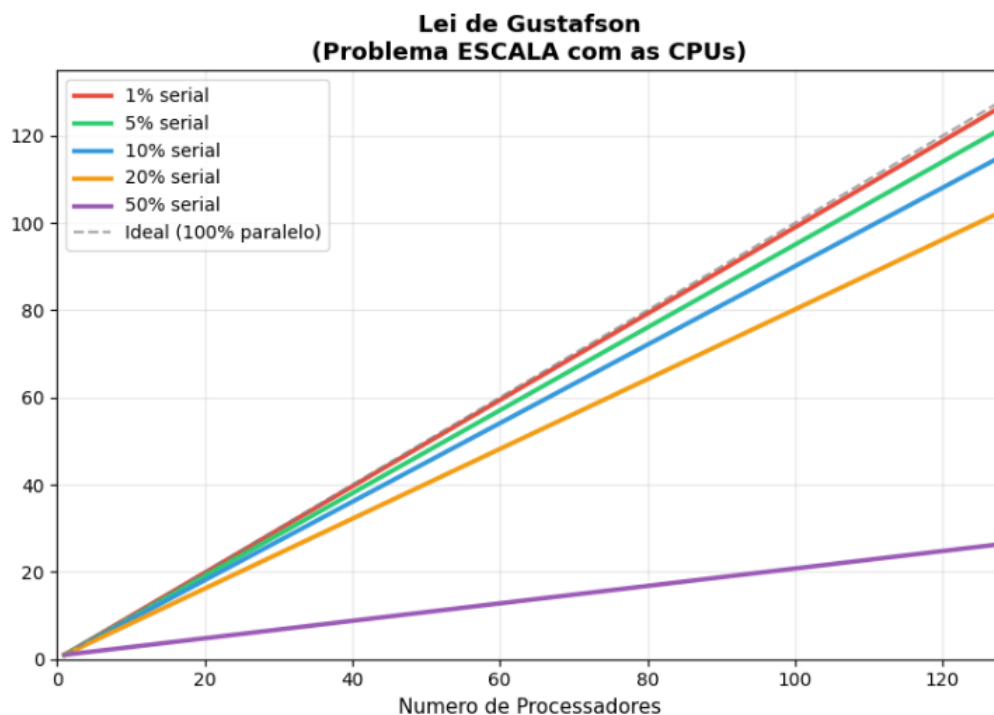
À medida que o número de recursos computacionais cresce indefinidamente (N para infinito), o termo P / N aproxima-se de zero, significando que o speedup máximo fica estritamente limitado pelo inverso da fração serial $1/(1 - P)$, enquanto a eficiência decai para zero (E para 0), se tratando assim de uma visão pessimista, pois mesmo, que adicionemos mais núcleos a um problema fixo, teremos retornos decrescentes severos devido ao gargalo da parte serial.

Lei de Gustafson: A Perspectiva de Carga Escalável Proposta por John Gustafson e Edwin Barsis em 1988, a Lei de Gustafson flexibiliza a premissa de Amdahl, argumentando que, na prática em um sistema computacional real, o tamanho do problema não permanece estático como é citado na Lei de Amdahl, e sim é crescente, e para melhorar a eficácia e velocidade do programa, é expandido o tamanho do problema (aumentando o volume de dados, o nível de detalhe de uma simulação ou o tamanho de um modelo de IA) para ser resolvido no mesmo intervalo de tempo original (GUSTAFSON, 1988). Fórmula do Speedup de Gustafson O ganho de velocidade escalado, tenta assumir um comportamento linear em relação ao número de recursos:

$$S = (1 - P) + P \cdot N \quad (4)$$

P = Tempo gasto na parte paralela; N = Número de processadores utilizado.

Figura 4 – Curva teórica da Lei de Gustafson: speedup escalado $S(N)$ em função do número de processadores N para diferentes valores da fração paralela P (50%, 75%, 90%, 95%). Diferente de Amdahl, o crescimento é aproximadamente linear, refletindo a hipótese de problema escalável



Fonte: Elaborada pelo autor (Google Colab).

Fórmula de Eficiência de Gustafson Dividindo o speedup escalado pelo número de processadores N, resulta na eficiência correspondente:

$$E(N) = \frac{S(N)}{N} = \frac{1 - P}{N} + P \quad (5)$$

Sendo assim ao contrário do modelo de Amdahl, que quando N tende ao infinito, a eficiência tende a ser zero, a eficiência na lei de Gustafson não decai ao zero, pois o termo $(1 - P / N)$ desaparece, fazendo com que a eficiência diminua para a própria fração paralela do sistema (E para P), por exemplo se 99% do tempo de execução de um problema expandido for em paralelo, a eficiência se manterá estável em próximo a 99%, o que viabiliza altos ganhos de velocidade mais “lineares” em computadores.

2.3 Hill & Marty (2008): Amdahl na era multicore

Este artigo (HILL; MARTY, 2008) demonstra uma forma abstrata de enxergar como a complexidade e organização de núcleos impacta o paralelismo e execuções que necessitam de baixa latência é feito o uso do termo BCE (equivalentes de núcleo base) uma forma de demonstrar a complexidade de um núcleo onde 1 BCE é o núcleo simples como a figura e explicação a seguir:

Figura 5 – Demonstração de 3 arquiteturas distintas utilizando o mesmo custo sendo o máximo de 16 BCE, observe que ambas arquiteturas possuem o mesmo custo e ocupam a mesma área

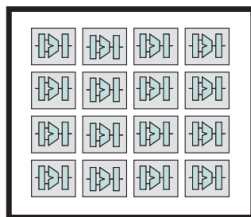


Figure 1. Symmetric
Sixteen 1-BCE cores

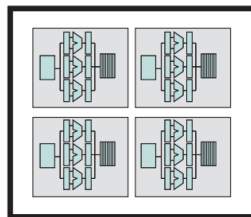


Figure 2. Symmetric
Four 4-BCE cores

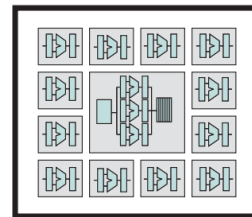


Figure 3. Asymmetric
One 4-BCE core, 12 1-BCE cores

Fonte: Adaptada de Hill e Marty (2008).

Arquitetura simétrica de 16 núcleos onde o custo de cada um é 1 BCE, pode ser associada a uma GPU onde seus núcleos seriam simplificados cenário onde o paralelismo pode ser mais viável conforme a complexidade da tarefa.

Arquitetura simétrica de 4 núcleos onde o custo de cada é 4BCE, pode ser associado a uma CPU onde há necessidade maior de núcleos complexos pois o foco dessa unidade é executar a operação no menor tempo possível e atividades com maior complexidade de desvio condicional, observe que ao aumentar a complexidade de um núcleo de CPU transistores são gasto em circuitos de predição, memórias como registradores e cache tais recursos.

Arquitetura Assimétrica 1 núcleo de 4 BCE e 12 núcleos de 1 BCE uma nova exploração para o mercado principalmente em computadores portáteis que necessitam possuir maior eficiência energética.

Todas as funções abaixo possuem as variáveis de entrada: f = fração paralela; n = orçamento total de BCE; r = a complexidade do núcleo dado em BCE. A função $perf(r)$ descreve a performance de um núcleo de tamanho r ; foi considerada a Lei de Pollack (POLLACK, 1999), onde $perf(r) = \sqrt{r}$.

$$Speedup_{Simétrico}(f, n, r) = \frac{1}{\frac{1-f}{perf(r)} + \frac{f \cdot r}{perf(r) \cdot n}} \quad (6)$$

Parte sequencial: $\frac{1-f}{perf(r)}$; parte paralela: $\frac{f \cdot r}{perf(r) \cdot n}$.

$$Speedup_{Assimétrico}(f, n, r) = \frac{1}{\frac{1-f}{perf(r)} + \frac{f}{perf(r) + n - r}} \quad (7)$$

Parte sequencial: $\frac{1-f}{perf(r)}$; parte paralela: $\frac{f}{perf(r) + n - r}$.

$$Speedup_{dinâmico}(f, n, r) = \frac{1}{\frac{1-f}{perf(r)} + \frac{f}{n}} \quad (8)$$

Parte sequencial: $\frac{1-f}{perf(r)}$; parte paralela: $\frac{f}{n}$.

2.4 Modelo Roofline (Williams, Waterman & Patterson, 2009)

Proposto por Williams, Waterman e Patterson em 2009 na Universidade de Berkeley (WILLIAMS; WATERMAN; PATTERSON, 2009), o modelo de Roofline surge como um modelo que requer uma parte visual para ser interpretado, com o objetivo de diagnosticar as limitações de desempenho em aplicações científicas em arquiteturas multicore e acelerações gráficas contemporâneas, que diferente dos métodos tradicionais por exemplo a Lei de Amdahl, por exemplo que isola parte do tempo de execução, no modelo de Roofline temos uma correlação direta, na qual a capacidade computacional de pico do hardware e o tráfego de dados gerado na aplicação pelo barramento de memória principal.

E seu parâmetro principal é a Intensidade Operacional (OI), que quantifica a densidade de processamento do algoritmo, ela expressa a razão entre o total de operações aritméticas do ponto flutuante (FLOPs) sendo executadas pelo algoritmo e o volume total de dados transferidos

(leituras e escritas) entre os registradores/caches do processador e a memória principal (RAM), medida em bytes:

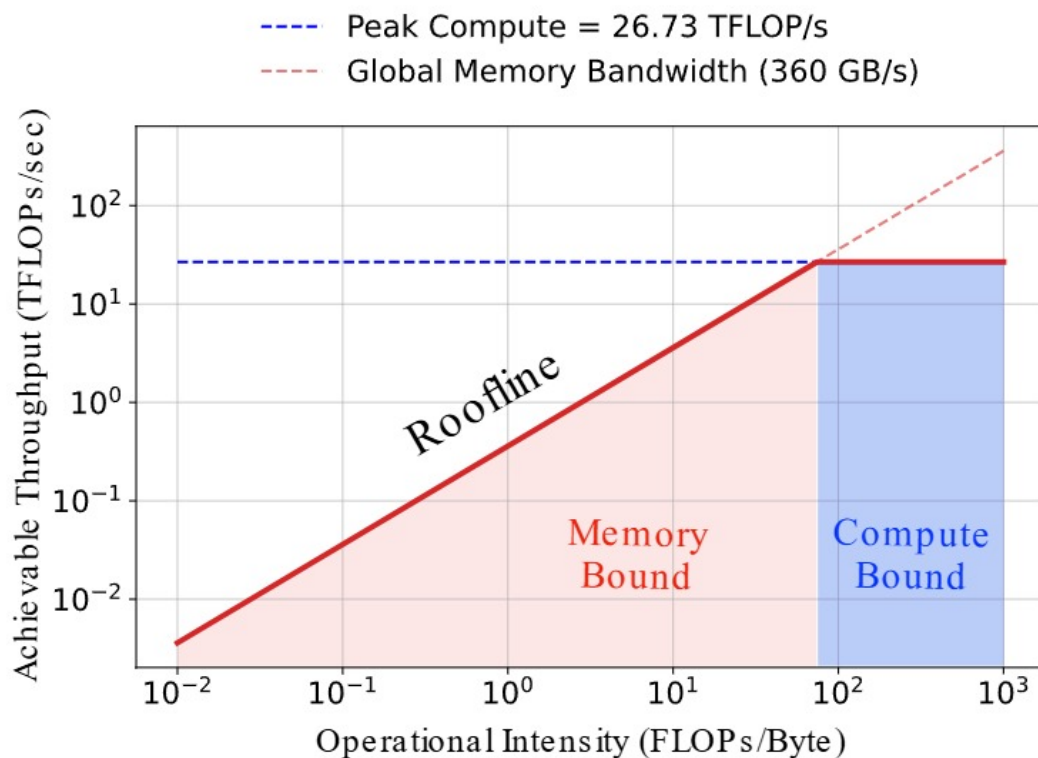
$$\text{Intensidade Operacional (OI)} = \frac{\text{Operações de Ponto Flutuante (FLOPs)}}{\text{Tráfego de Memória (Bytes)}} \quad (9)$$

Já visualmente essa equação projeta um perfil com formato de teto, que divide o espaço de execução em duas regiões críticas (bottleneck):

Região Limitada por Memória (Memory-Bound): visualmente é representada pela rampa inclinada no gráfico, a mesma ocorre quando a Intensidade Operacional da aplicação é baixa, ou seja, quando poucos cálculos são realizados por byte transferido, e percebemos que o desempenho do sistema fica estritamente condicionado pela taxa de transferência da memória (GB/s), significando que os núcleos de processamento passam ciclos ociosos à espera dos dados serem trazidos da RAM.

Compute-Bound: é representada no gráfico pela linha horizontal superior, e ocorre quando o algoritmo possui uma elevada Intensidade Operacional, e tendo o barramento de memória deixando de ser um fator limitante, e a aplicação atinge a saturação de processamento, deixando o ganho de desempenho restrito exclusivamente ao poder matemático bruto (frequência de clock, unidades vetoriais SIMD e quantidade de núcleos) da CPU ou GPU.

Figura 6 – Modelo Roofline: o teto de computação (Peak Compute) e o teto de memória (largura de banda) delimitam as regiões memory-bound e compute-bound conforme a Intensidade Operacional do algoritmo. O joelho entre as duas regiões ocorre na $OI \approx 74$ FLOPs/byte ($26,73 \text{ TFLOP/s} \div 360 \text{ GB/s}$)



Fonte: Accelerated Computing Academy (2025). Disponível em:
<<https://accelerated-computing.academy/fall25/resources/roofline/>>

3 METODOLOGIA

A pesquisa é de natureza experimental e quantitativa. Foram utilizados dois grupos de algoritmos os de busca onde possui dependência forte em acesso à memória fazendo-se pouco uso de unidades internas de processamento e os de matemática densa onde o peso está em cálculos fazendo-se maior uso de unidades interna do processador, cada algoritmo foi implementado em três alvos de execução, CPU single-core (1 thread), CPU paralela (OpenMP, N threads) e GPU (CUDA muitas threads) nos algoritmos de busca foi submetidos a volumes crescentes de dados e para algoritmos de matemática densa foram submetidos a quantidades de pontos maiores Monte Carlo e Mandelbrot onde o maior número de pontos representa maior volume de cálculos, para cada execução mediu-se o tempo de execução no qual derivam o speedup e a eficiência e em seguida confrontado com as previsões das Leis de Amdahl e Gustafson.

3.1 Desenho experimental

São os três fatores controlados:

- Alvo de execução: serial, OpenMP ou CUDA;
- Número de threads (OpenMP): varrido em potências de 2, 1,2,4,8 ... até a quantidade máxima de threads lógicas disponíveis da máquina;
- Volume de dados (N a quantidade de estrutura de `event` (ver 3.3) dentro do array);
- O volume de pontos usado nos algoritmos de matemática densa.

E três as grandezas medidas:

- **Tempo de execução** (mediana, em segundos);
- **Speedup** = tempo serial $\div$ tempo do alvo paralelo;
- **Eficiência** = speedup $\div$ número de processadores (threads, no caso do OpenMP).

3.2 Parâmetros do benchmark e recursos por máquina

Os experimentos rodaram em três máquinas com perfis distintos de paralelismo, da faixa de 8 a 16 threads de CPU e de 896 a 8.704 núcleos CUDA:

O código, escrito em C (padrão C99), foi compilado em modo otimizado (Release) pela toolchain **MSVC** (Visual Studio, prompt *x64 Native Tools*): a parte de CPU usou o **cl.exe** com **OpenMP** (/openmp) e otimização /O2; os kernels de GPU usaram **nvcc** (com cl.exe como host compiler) e otimização -O3. O executável CUDA é gerado como binário **multiarquitetura**,

Tabela 1 – Recursos por máquina

Máquina	CPU	Núcleos / Threads	GPU	CUDA cores	RAM / SO
1	Ryzen 7 4800H	8c / 16t	GTX 1650	896	8 GB / Windows
2	Intel i5 12400	6c / 12t	RTX 3060	3.584	32 GB / Windows
3	Intel i3 10105F	4c / 8t	RTX 3080	8.704	8 GB / Windows

Fonte: Elaborada pelo autor.

cobrindo as gerações Turing (sm_75, GTX 1650) e Ampere (sm_86, RTX 3060 e RTX 3080). O executável foi **compilado uma única vez** e distribuído às três máquinas junto com as runtimes do MSVC (vcomp140.dll, vcruntime140.dll); por ser multiarquitetura, roda de forma nativa em cada GPU sem recompilação. Versões do ambiente de compilação: Visual Studio 2022 Community (Developer Command Prompt v17.9.6, x64) como host `cl.exe`, e CUDA Toolkit nvcc release 13.2, V13.2.78 (build cuda_13.2.r13.2/compiler.37668154_0).

3.3 Geração de dados

Os dados de entrada são **eventos sintéticos**: cada evento carrega um valor gerado pelo método de **Box-Muller** (distribuição normal), simulando a leitura de um sensor genérico. A implementação usa a variante **polar** de Box-Muller (por rejeição, sem $\cos/\sin$), com o gerador uniforme `rand()` da biblioteca padrão semeado por `srand(42)`; a geração roda uma única vez na CPU, e o mesmo array é entregue aos três alvos (serial, OpenMP e CUDA). Os volumes testados em algoritmos de busca foram `volumes_busca=1000, 100000, 1000000, 10000000, 20000000` todos sendo do tipo `Event` que ainda será abordado, a geração usa **semente fixa**, de modo que o mesmo conjunto de dados é reproduzido em toda execução garantindo que as comparações entre alvos e máquinas partam de entradas idênticas, para algoritmos de matemática densa foi utilizado `volumes_math=1000000, 5000000, 10000000, 50000000, 100000000`, que representa a quantidade de pontos usado nos algoritmos.

A estrutura abaixo chamada `Event` é o modelo de um único evento, sendo utilizados por um array de `N` tamanho conforme definido `volumes_busca`.

```

1  typedef struct {
2      uint32_t    id;
3      int64_t     timestamp;
4      float       valor;
5      float       valor_secundario;
6      Categoria   categoria;
7      uint8_t     status;
8      char        tag[32];
9      char        origem[16];
10     uint32_t     checksum;
11     uint8_t      _pad[8];
12 } Event;

```

Código 1 – Estrutura Event (96 bytes)

O campo **valor** é o que recebe a leitura gerada por Box-Muller e sobre o qual as buscas filtram.

O protocolo de cronometragem é o mesmo para todos os algoritmos. Antes de cada medição, uma execução de **warm-up** é descartada (para aquecer a cache); em seguida a operação é repetida nove vezes $n_repeticoes=9$ e adota-se a **mediana** como tempo final escolhida deliberada por ser mais robusta a picos esporádicos do sistema operacional do que a média. O relógio utilizado é monotônico, com precisão de nanosegundos.

No alvo **CUDA**, o tempo total é decomposto em duas parcelas medidas separadamente: o tempo de **transferência pela PCIe** (cópia da DRAM para a memória da GPU VRAM) e o tempo de **kernel** (execução na GPU). Essa separação é central para interpretar o speedup da GPU nas buscas, onde a transferência frequentemente domina o tempo total.

3.4 Limites

Duas delimitações afetam os algoritmos de busca:

- O tempo medido da **busca binária não inclui a pré-ordenação** ($O(n \log n)$), executada uma única vez fora do cronômetro;
- O tempo da **busca por hash não inclui a construção da tabela**.

Ambos são tratados como custo de preparação amortizado. É uma escolha que **favorece** esses dois algoritmos, sob uma consulta única o custo de preparação os tornaria inviáveis, mas esta alinhado com o objetivo de medir o comportamento de paralelização da operação de busca.

4 DESENVOLVIMENTO

Esta seção apresenta os algoritmos implementados em duas classes diferentes: os de busca, excelentes para demonstrar o memory-bound, e os de matemática densa, excelentes para

demonstrar o compute-bound. Toda essa implementação evidencia os limites do paralelismo e os fatos a se considerar, como a forma que a arquitetura do computador influencia o desempenho. Ao terminar a leitura desta seção, o leitor consegue construir um raciocínio sobre as várias propriedades que impactam a computação, descritas aqui ou em outras fontes.

4.1 Algoritmos de busca

Nesta classe de algoritmos são explorados problemas de memória, em que toda operação depende mais do acesso ao conjunto de dados do que da computação de operações lógicas, aritméticas e de ponto flutuante. É a classe que explora principalmente o memory-bound.

Tabela 2 – Algoritmos de busca e a lei que cada um ilustra

Algoritmo	Big-O (serial)	Limitado por	Lei que ilustra
Busca Linear	$O(n)$	memória (banda)	Amdahl + Gustafson
Busca Binária	$O(\log n) + O(k)$	overhead (setup/transfer)	Amdahl no limite
Hash Lookup	$O(1)$ amortizado	memória (acesso aleatório)	Amdahl + Gustafson

Fonte: Elaborada pelo autor.

4.1.1 Busca Linear

Realiza uma varredura em todo o array, buscando o(s) elemento(s) que correspondem ao parâmetro de busca. Sua complexidade é $O(n)$: à medida que a quantidade de elementos cresce, a quantidade de operações acompanha. Possui alta compatibilidade com paralelismo, pois cada elemento é independente e, por ser uma comparação simples, não necessita de núcleos complexos. O custo real está no acesso à memória para guardar os resultados e, no final, mesclá-los, sendo essa a parte sequencial. É aí que o paralelismo deve tomar cuidado: no acesso e na gravação em memória.

```

1  SearchResult busca_linear_serial(Event *dados, int n, float min, float
   ↪ max) {
2      Event **encontrados = (Event **)malloc((size_t)n * sizeof(Event *));
3      int count = 0;
4      for (int i = 0; i < n; i++) {
5          if (dados[i].valor >= min && dados[i].valor <= max) {
6              encontrados[count++] = &dados[i];
7          }
8      }
9      SearchResult r = { encontrados, count };
10     return r;
11 }

```

Código 2 – Busca linear serial (1 thread)

observe o for na medida que o argumento n aumenta a quantidade de execução do for aumenta junto.

```
1 SearchResult busca_linear_openmp(Event *dados, int n, float min, float
  ↳ max, int threads) {
2     Event **encontrados = (Event **)malloc((size_t)n * sizeof(Event *));
3     int *counts = (int *)calloc((size_t)threads, sizeof(int));
4     int *offsets = (int *)calloc((size_t)threads, sizeof(int));
5     Event ***temp = (Event ***)malloc((size_t)threads * sizeof(Event
  ↳ **));
6     for (int t = 0; t < threads; t++)
7         temp[t] = (Event **)malloc((size_t)n * sizeof(Event *));
8     int i;
9     #pragma omp parallel for schedule(static)
10    for (i = 0; i < n; i++) {
11        if (dados[i].valor >= min && dados[i].valor <= max) {
12            int tid = omp_get_thread_num();
13            temp[tid][counts[tid]++] = &dados[i];
14        }
15    }
16
17    int total = 0;
18    for (int t = 0; t < threads; t++) { offsets[t] = total; total +=
  ↳ counts[t]; }
19    for (int t = 0; t < threads; t++) {
20        memcpy(&encontrados[offsets[t]], temp[t], (size_t)counts[t] *
  ↳ sizeof(Event *));
21        free(temp[t]);
22    }
23    free(temp); free(counts); free(offsets);
24    SearchResult r = { encontrados, total };
25    return r;
26 }
```

Código 3 – Busca linear em OpenMP (buffer privado por thread + merge)

Event ***temp é o armazenamento temporário usado durante a execução da função. ***temp = [tid][linha] é uma matriz que guarda endereços em duas dimensões: o tid é o identificador único da thread, gerado pela função omp_get_thread_num(), que retorna um número de 0 a N-1. Ou seja, cada thread possui memória exclusiva para trabalhar (essencial para evitar erros), e a linha é onde de fato a thread armazena os endereços válidos para a condição de busca.

Exemplo com 5 threads varrendo um array dados de 500 eventos (cada thread pega um bloco de 100, via schedule(static)). Cada coluna é o caderninho de uma thread; cada célula guarda o **endereço** de um match que ela achou no bloco dela:

Endereços **ilustrativos** (dados começa em ~0x55F0A000; cada thread varre um bloco contíguo diferente, por isso os endereços de uma mesma coluna ficam próximos e saltam entre colunas):

Repare: cada thread acha uma quantidade **diferente** de matches (counts[tid]), e nunca escreve na coluna de outra, sem condição de corrida. No fim, o merge concatena as colunas

Tabela 3 – Buffer privado por thread (temp[tid][linha]): endereços ilustrativos

Thread Id →	0	1	2	3	4
linha 0	0x55F0A120	0x55F0D2C0	0x55F11A40	0x55F143F0	0x55F17C10
linha 1	0x55F0A4B0	0x55F0D880	0x55F11E10		0x55F17F90
linha 2	0x55F0A930		0x55F12150		0x55F18320
linha 3			0x55F124E0		
counts[tid]	3	2	4	1	3

Fonte: Elaborada pelo autor.

em ordem (T0, depois T1, ...) num único array, essa concatenação é a parte sequencial. Esse conceito se repetira em outros algoritmos.

```

1  int i = blockIdx.x * blockDim.x + threadIdx.x;
2  if (i < n && dados[i].valor >= vmin && dados[i].valor <= vmax) {
3      int pos = atomicAdd(d_count, 1);
4      out_indices[pos] = i;
5  }
```

Código 4 – Trecho CUDA da busca linear: a GPU resolve a corrida com atomicAdd

Limitado pelo tempo de transferência de barramento, expõe que o tempo de copia do array de eventos da Dram para a Vram pode ser maior que o tempo de execução dentro da GPU, o merge final e a parte sequencial Lei Amdahl (o merge é a fração serial que limita o ganho) e Gustafson (crescendo N, a complexidade do problema domina e o peso do merge encolhe).

4.1.2 Busca Binária

Exige um array ordenado e, a cada operação, corta o espaço de busca pela metade. Sua complexidade é descrita como $O(\log n)$ para localizar e $O(k)$ para coletar os k resultados no intervalo. Demonstra a relação entre a divisão de trabalho e a parte paralelizável: é o caso em que dividir o trabalho pesa mais do que executá-lo sem divisão. Custo inicial de ordenação $O(n \log n)$ e roda de forma serial (qsort) para fim de simplificar não está sendo considerado o intuito e mostrar como um algoritmo eficiente para resolver o problema se comporta sendo paralelizável.

Demonstra na lei do Amdahl onde a fração paralelizável P se torna próximo a 0, na medida que aumenta N numero de processadores o speedup fica próximo a 1, ou seja na media que um algoritmo se possui a maior parte de fração serial forçar o paralelismo no nível de algoritmos pode ter o efeito oposto visto que possui tempo de divisão de trabalho e mesclagem de resultados.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}} \quad (10)$$

OpenMP, mesmo padrão da busca linear: cada thread faz busca binária no seu bloco e grava no buffer próprio (`temp[t]`); a junção final é serial (a fração de Amdahl). Código completo no repositório ([PEREIRA, 2026]).

```
1  cudaMemcpy(d_dados, dados, n * sizeof(Event), cudaMemcpyHostToDevice);
```

Código 5 – Trecho CUDA da busca binária: a transferência H2D domina

O tempo gasto para copiar um array de eventos da DRAM para a VRAM, trafegando pelo barramento PCIe, é o ponto crítico de executar uma operação de busca em uma GPU. Se esse tempo fosse desconsiderado, a GPU conseguiria superar a CPU em volumes de dados maiores, conforme a Lei de Gustafson, já que cada comparação é uma operação simples. Se fossem adicionadas operações complexas a essa busca, como matemática profunda, a GPU poderia exibir speedup maior já em volumes de dados menores.

4.1.3 Hash Lookup

É uma tabela que guarda cada evento em uma “gaveta” (bucket). A identificação do bucket é um número calculado a partir do `id` do `Event`, por meio de uma função hash que retorna um valor entre 0 e 131101; o bucket é definido por essa posição, e mais de um `id` pode cair na mesma gaveta. O código abaixo mostra a função hash. O cronometro não considera a construção da tabela hash que é executada uma vez.

```
1  #define HASH_TAMANHO 131101
2
3  static inline int hash_fn(uint32_t key, int tamanho) {
4      return (int)((key * 2654435761u) % (uint32_t)tamanho);
5  }
```

Código 6 – Função hash do Hash Lookup

Para achar um evento não se varre nada: calcula-se a gaveta e acessa-se exatamente o endereço dela, com custo $O(1)$ no melhor caso, sem colisões. A intenção é que haja colisões durante o benchmark. A colisão (dois ou mais `ids` caindo na mesma gaveta) é resolvida com uma lista encadeada, o que expõe o problema de cache miss, pois cada elemento da lista encadeada pode estar em um local de memória diferente. Além disso, não há paralelismo, pois é necessário ler o primeiro elemento para ter acesso ao próximo.

OpenMP (o gargalo é o acesso aleatório à memória):

Dividir o trabalho em n threads, neste algoritmo, expõe ainda mais o problema de cache miss, pois o evento possivelmente não estará disponível em cache. Cada execução individual é totalmente sequencial; porém, ao rodar um conjunto de várias buscas de hash lookup, é benéfico

ter mais threads. Se não é possível reduzir o tempo mínimo de execução individual, como propõe Amdahl, então, ao aumentar a complexidade do problema, pode-se observar maior speedup, o comportamento de Gustafson.

O código abaixo mostra a distribuição de `n_ids` para várias threads; o restante do algoritmo é totalmente sequencial e independente, no sentido de que uma busca não depende de outra.

```
1 #pragma omp parallel for schedule(static)
2 for (i = 0; i < n_ids; i++) {
3     int idx = hash_fn(ids[i], ht->tamanho);
4     for (HashNode *cur = ht->buckets[idx]; cur; cur = cur->prox)
5         if (cur->key == ids[i]) { encontrados[i] = cur->evento; break; }
6 }
```

Código 7 – Hash Lookup em OpenMP (o gargalo é o acesso aleatório à memória)

CUDA (a lista encadeada precisa ser “achatada” antes do H2D): aqui o problema de memória volta a incomodar: como cada bucket pode ter listas encadeadas em regiões de memória distintas, há um gasto extra para “achatar” (empacotar) as listas encadeadas antes do H2D.

O código-fonte completo do benchmark, com todos os algoritmos em C (serial), OpenMP e CUDA, está disponível no repositório ([PEREIRA, 2026]).

4.2 Algoritmos de matemática densa

Ao contrário da classe de algoritmos de busca, esta classe explora o compute-bound: cada unidade de trabalho é um cálculo de ponto flutuante puro, com acesso mínimo à memória. Crescer o problema o escala de forma quase linear (o caso ideal para mostrar Gustafson), e ainda é possível observar como a complexidade do núcleo de processamento interfere.

4.2.1 Monte Carlo de π

Joga N pontos aleatórios (x, y) em $[0, 1)^2$ e conta quantos caem no quarto de círculo ($x^2 + y^2 \leq 1$); $\pi \approx 4 \times (\text{dentro}/N)$. Cada ponto é cálculo puro e independente aumentar N (mais precisão) só aumenta a parte paralela e a parte serial (a redução final) permanece desprezível. Por isso adicionar mais núcleos de processamento continua melhorando o speedup: este é o caso ideal da Lei de Gustafson, em que o ganho escala quase linear com o número de threads.

$$S = (1 - P) + P \cdot N \quad (11)$$

OpenMP, `reduction(+:inside)`, sequência LCG por thread: `reduction(+:inside)` é a instrução que dá a cada thread um contador pessoal, resolvendo o problema de acesso e

gravação em memória, uma tarefa indivisível. No final da execução, realiza-se uma operação sobre esses contadores; neste caso, o argumento passado é de soma, sendo essa a fração serial.

```
inside vale 100
Thread A: le 100  -----+
Thread B: le 100      |    (B leu antes de A escrever!)
Thread A: soma -> 101 |
Thread B: soma -> 101 |
Thread A: escreve 101 |
Thread B: escreve 101 +-  inside agora vale 101 - mas DEVIA valer 102
```

Observe como o código abaixo possui poucas entradas, diferente dos algoritmos de busca, em que é necessário acessar o array completo de N eventos, pesando muito no cache. O código a seguir mostra a dependência mínima de acesso à memória: a maior parte dos dados é gerada pelo próprio algoritmo, concentrando a maior parte das operações nas unidades aritméticas e de ponto flutuante, sendo esse o ponto de maior peso, o compute-bound.

```
1  double montecarlo_omp(long n, int threads) {
2  #define MC_BASE_SEED  0xDEADBEEFCAFEBAEULL
3  #define MC_LARGE_PRIME 6364136223846793005ULL
4      long inside = 0;
5  #ifdef _OPENMP
6      #pragma omp parallel num_threads(threads) reduction(+:inside)
7      {
8          int tid = omp_get_thread_num();
9          uint64_t state = MC_BASE_SEED ^ ((uint64_t)(unsigned)tid *
          ↪ MC_LARGE_PRIME);
10         long local_n = n / (long)threads;
11         if (tid == threads - 1) local_n = n - local_n * (long)(threads
          ↪ - 1);
12         long local_inside = 0;
13         for (long i = 0; i < local_n; i++) {
14             double x = lcg_double(&state);
15             double y = lcg_double(&state);
16             if (x * x + y * y <= 1.0) local_inside++;
17         }
18         inside += local_inside;
19     }
20 #else
21     (void) threads;
22     return montecarlo_serial(n);
23 #endif
24     return 4.0 * (double)inside / (double)n;
25 }
```

Código 8 – Monte Carlo de pi em OpenMP (reduction + sequência LCG por thread)

CUDA (a junção é uma redução em árvore na shared memory):

```

1  __global__ void kernel_montecarlo(long n, unsigned long long *d_inside)
    ↪ {
2      extern __shared__ unsigned long long sdata[];
3      int gid = blockIdx.x * blockDim.x + threadIdx.x;
4      int stride = blockDim.x * blockDim.x;
5      int tid = threadIdx.x;
6      unsigned int state = (unsigned int)(gid * 1664525u + 1013904223u) ^
    ↪ 0xDEADBEEFu;
7      unsigned long long local_inside = 0;
8      for (long i = (long)gid; i < n; i += (long)stride) {
9          float x = lcg32_float(&state);
10         float y = lcg32_float(&state);
11         if (x * x + y * y <= 1.0f) local_inside++;
12     }
13     sdata[tid] = local_inside;
14     __syncthreads();
15     for (int s = blockDim.x / 2; s > 0; s >>= 1) {
16         if (tid < s) sdata[tid] += sdata[tid + s];
17         __syncthreads();
18     }
19     if (tid == 0) atomicAdd(d_inside, sdata[0]);
20 }

```

Código 9 – Kernel CUDA do Monte Carlo (grid-stride loop + redução em árvore)

A complexidade de organização da GPU é maior, sendo **grid** → **blocos** → **threads**. `sdata` é a alocação de memória compartilhada na VRAM, declarada com `extern __shared__`; as outras variáveis, que não têm essa declaração, são privadas por thread. Depois de realizado o cálculo, a função `__syncthreads()`; sincroniza todas as threads, aguardando que finalizem. A próxima instrução:

```

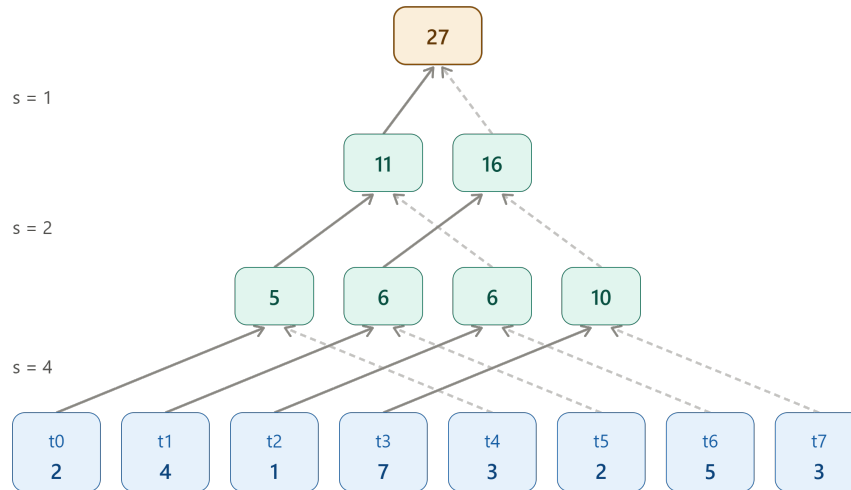
1      for (int s = blockDim.x / 2; s > 0; s >>= 1) {
2          if (tid < s) sdata[tid] += sdata[tid + s];
3          __syncthreads();
4      }

```

Código 10 – Redução em árvore na shared memory

Faz uma redução em árvore onde existe uma correspondência exemplo 8 threads `[0, 1, 2, 3, 4, 5, 6, 7]` e seus dados respectivos `[2, 4, 1, 7, 3, 2, 5, 3]` mostrado na figura abaixo:

Figura 7 – Exemplo da redução em árvore executada de forma paralela, calculando o resultado final do trabalho realizado pelo conjunto de threads referentes àquele bloco



Fonte: Elaborada pelo autor.

o valor final e adicionado a uma variável sendo o resultado total de todos blocos if (tid == 0) atomicAdd(d_inside, sdata[0]);.

4.2.2 Mandelbrot 2D

O mandelbrot gera uma imagem onde cada pixel é um ponto c do plano complexo. Para cada pixel se repete a conta $z = z^2 + c$ (começando em $z = 0$) e é contando quantas vezes ele roda até duas coisas acontecer:

- **z escapa** ($|z|^2 > 4$): o ponto fica **fora** do conjunto (escapa rápido, poucas iterações);
- **bate o teto de 256 repetições**: o ponto fica **dentro** do conjunto (vai até o fim, caro).

Ou seja pixels diferentes levam números de iterações diferentes surgindo um desbalanceamento de carga.

OpenMP, a linha que importa é o `schedule` (por causa do desbalanceamento):

```
1 #pragma omp parallel for num_threads(threads) reduction(+:total)
   ↪ schedule(dynamic, 4)
```

Código 11 – Diretiva OpenMP do Mandelbrot (schedule dinâmico)

No Mandelbrot com OpenMP, foi necessário deixar a divisão de iterações por conta do OpenMP: cada thread pega 4 iterações, resolve e, ao finalizar, busca mais 4. Isso se deve à variação da carga de trabalho, em que os pixels têm custos diferentes.

CUDA (o efeito colateral é divergência de warp): diferente do openMp não existe formula de divisão de carga de tarefas, cada thread já é responsável por um pixel, por exemplo pixel 100 entra na thread N 99 e assim por diante, nesse mesmo bloco as threads rápidas aguardam as lentas isso é chamado de divergência de warp uma warp e um grupo de threads indivisível as threads que finalizaram devem aguardar todo o seu wrap finalizar antes de pegar a próxima tarefa.

5 RESULTADOS E DISCUSSÃO

Todos os resultados gerados pelo benchmark se encontram disponíveis no repositório no GitHub. Primeiro são apresentados os tempos e speedups medidos; logo depois vem a interpretação por intensidade operacional. O artigo de projeto (etapa 1) ([PEREIRA; FAJARDO, 2026) declarou 3 hipóteses (H1/H2/H3) mais os resultados esperados quantitativos. Nesta Seção 5, cada uma recebe sua classificação de validação: **confirmada / parcial / refutada**.

- H1 (busca linear ganha mais speedup com o aumento do volume de dados) confirmada, com correção. Em volume pequeno, o speedup declina por causa do overhead de criação de threads; a partir de certo volume, converte-se em ganho, porém não escala livremente: ao aumentar o volume de dados, aumenta a dependência de acesso à memória. As propriedades da memória (como a velocidade de transferência) são o fator limitador pois a intensidade operacional está em memory-bound e o processador encontra-se fazendo operação logica simples.
- H2 (as colisões do hash viram a fração serial de Amdahl) parcialmente confirmada. As colisões de hash se tornam lista com elementos encadeados sendo sim uma perda de eficiência, dificilmente uma colisão pode ser tratada com busca binária, o custo para ordenar o array onde contém os registros pode ser inviável em um sistema que insere muitos elementos a solução e ser uma busca sequencial quando há ocorrência de colisão, e na medida que a porcentagem de colisões ultrapassa um percentual tolerado e melhor fazer gastar recursos para recriar uma tabela hash com mais buckets, outro fator interessante e o acesso aleatório à memória: como o hash chega direto a um endereço, sem varrer o caminho, a memória cache não está preparada e ocorre cache miss. Isso demonstra a fragilidade de acesso direto a um endereço que ainda nem foi alocado.
- H3 (algoritmos de matemática densa favorecem a GPU) confirmada com acréscimo de correção. O tempo de PCIe não se torna desprezível pelo volume de cálculos, ele se torna desprezível poque a transferência de dados é mínima e o tempo de divisão de trabalho se tornam viáveis quando o maior peso está na complexidade do cálculo, ou seja, quando se usam as unidades de cálculo e lógica do processador com poucos dados de entrada, reduzindo a dependência de acesso à memória. Houve um caso em que as unidades internas da GPU realizaram o cálculo tão rápido que a transferência PCIe voltou a pesar: é o caso

da Máquina 3, mostrando que ela suporta uma intensidade operacional maior comparado com as outras máquinas.

5.1 Análise dos dados

As Tabelas 4, 5 e 6 trazem os resultados gerados pelo benchmark.

Tabela 4 – Tempo e speedup por alvo (Máquina 1: Ryzen 7 4800H + GTX 1650)

Algoritmo	Vol	Serial (ms)	OMP (ms)	S _{OMP}	CUDA (ms)	S _{CUDA}	Classe
Busca binária	20M	4,58	4,94 (T8)	0,93 ×	413,23	0,011 ×	memory-bound
Busca linear	20M	190,14	107,84 (T16)	1,76×	406,80	0,47 ×	memory-bound
Hash lookup	20M	81,82	14,88 (T16)	5,50×	59,61	1,37×	misto
Monte Carlo π	100M	216,27	34,19 (T16)	6,33×	16,08	13,45 ×	compute-bound
Mandelbrot	100M	11 553,78	976,71 (T16)	11,83×	70,73	163,34 ×	compute-bound

Fonte: Elaborada pelo autor.

Tabela 5 – Tempo e speedup por alvo (Máquina 2: i5-12400 + RTX 3060)

Algoritmo	Vol	Serial (ms)	OMP (ms)	S _{OMP}	CUDA (ms)	S _{CUDA}	Classe
Busca binária	20M	2,57	2,70 (T2)	0,95 ×	190,07	0,014 ×	memory-bound
Busca linear	20M	175,31	71,10 (T12)	2,47×	188,24	0,93 ×	memory-bound
Hash lookup	20M	84,24	12,89 (T12)	6,53×	27,23	3,09×	misto
Monte Carlo π	100M	219,27	26,77 (T12)	8,19×	17,76	12,35 ×	compute-bound
Mandelbrot	100M	11 374,06	1 188,52 (T12)	9,57×	56,99	199,59 ×	compute-bound

Fonte: Elaborada pelo autor.

Tabela 6 – Tempo e speedup por alvo (Máquina 3: i3-10105F + RTX 3080)

Algoritmo	Vol	Serial (ms)	OMP (ms)	S _{OMP}	CUDA (ms)	S _{CUDA}	Classe
Busca binária	20M	4,18	5,05 (T2)	0,83 ×	371,19	0,011 ×	memory-bound
Busca linear	20M	180,54	120,82 (T8)	1,49×	370,32	0,49 ×	memory-bound
Hash lookup	20M	64,30	20,33 (T8)	3,16×	26,46	2,43×	misto
Monte Carlo π	100M	249,29	65,29 (T8)	3,82×	3,65	68,37 ×	compute-bound
Mandelbrot	100M	15 534,11	2 246,53 (T8)	6,91×	58,34	266,27 ×	compute-bound

Fonte: Elaborada pelo autor.

Os cinco algoritmos não respondem ao paralelismo da mesma forma. Cada um tem uma fração paralela (onde é possível dividir o trabalho entre threads e reduzir o tempo de execução) e uma fração serial, onde existem dependências sequenciais indivisíveis. Alguns ganham velocidade até certo número de threads e depois estacionam; outros continuam acelerando à medida que se adicionam mais núcleos. Medir algoritmos diferentes com o mesmo objetivo, classificados em busca e matemática densa, não é acidente: é a forma de expor o que cada carga faz com a

memória. Invertendo a Lei de Amdahl (AMDAHL, 1967) sobre o speedup medido, estima-se a fração paralelizável P de cada algoritmo e prevê-se onde cada um deveria saturar.

Tabela 7 – Fração paralela (P) e serial ($1-P$) inferida por algoritmo (inversão da Lei de Amdahl sobre o speedup medido)

Algoritmo	Fração paralela P (inferida)	Fração serial ($1-P$)	Classe
Busca binária	$\sim 0\%$ (próxima de 0)	$\sim 100\%$	memory-bound
Busca linear	$\sim 38-65\%$	$\sim 35-62\%$	memory-bound
Hash lookup	$\sim 78-92\%$	$\sim 8-22\%$	misto
Monte Carlo π	$\sim 84-96\%$	$\sim 4-16\%$	compute-bound
Mandelbrot	$\sim 98\%$	$\sim 2\%$	compute-bound

Fonte: Elaborada pelo autor.

As faixas refletem a variação entre as três máquinas. Repare no contraste: a busca binária é quase toda serial (não há trabalho suficiente para dividir), enquanto Mandelbrot e Monte Carlo são quase totalmente paralelos, é essa fração que decide se o algoritmo satura cedo (Amdahl) ou escala com os recursos (Gustafson).

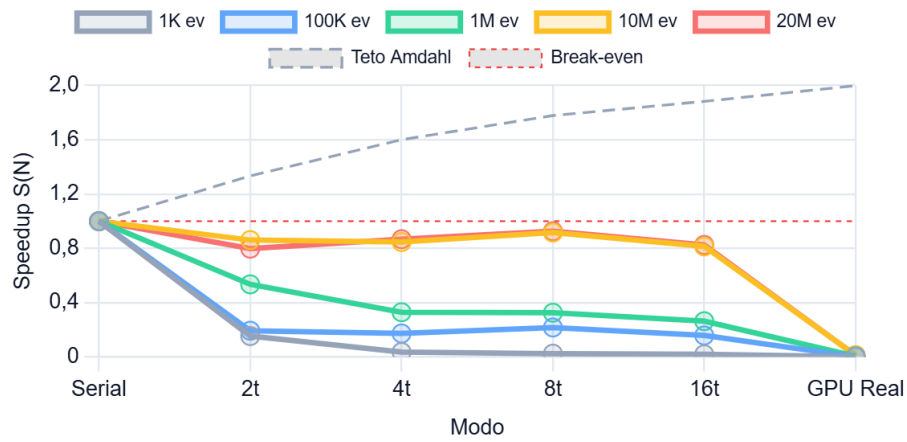
A busca linear é o caso mais próximo do Amdahl clássico. Seu speedup cresce com as threads, mas estabiliza em torno de $2\times$ e não sobe mais, mesmo adicionando núcleos. A fração paralela inferida ($\sim 38-65\%$ dependendo da máquina) coloca o teto de Amdahl entre $1,6\times$ e $2,8\times$, exatamente onde se observa que adicionar mais núcleos não melhora o speedup. Aqui a fração serial não é apenas código rodando de forma sequencial: é a limitação do tráfego de memória nos barramentos. A CPU ainda tem muito poder de processamento disponível, então o limite não está no cálculo, e sim na memória. Por isso a necessidade de medir a intensidade operacional, que coloca a busca linear na região memory-bound do Modelo Roofline (Seção 2.4).

A busca binária demonstra que forçar o paralelismo pode sabotar o ganho e piorar o tempo de execução: o speedup fica abaixo de 1, ou seja, a versão paralela é mais lenta que a serial. O algoritmo toca pouquíssimos elementos, então não há trabalho suficiente para dividir, e o custo de criar e sincronizar threads cobra caro, tornando-se inviável. É o caso em que a fração paralela de Amdahl fica próxima de 0.

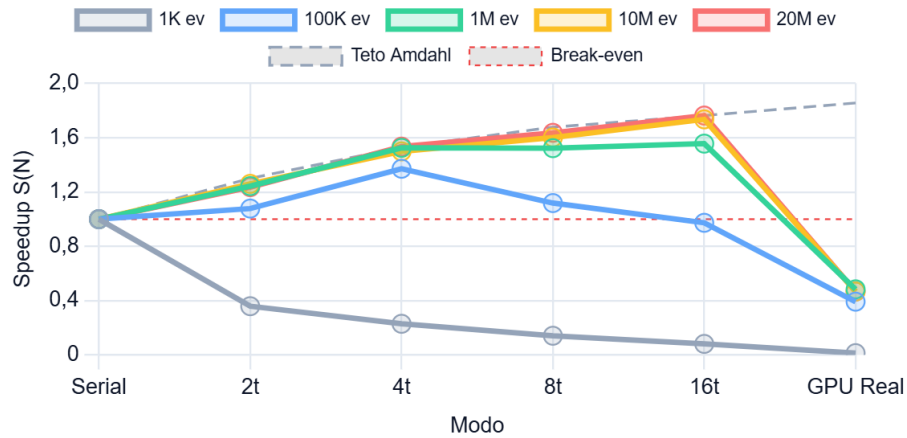
O hash lookup escala bem: sua fração paralela fica entre 78% e 92% dependendo da máquina. Mas o speedup medido fica abaixo do teto teórico, indicando que o potencial é dissipado por vários fatores, como a sincronização e o gargalo de cache. Como o hash descobre o endereço buscado de forma direta, na maior parte dos casos o dado não está em cache, causando muitos cache miss.

As figuras abaixo mostram visualmente os resultados dos algoritmos de busca, onde é possível observar que, ao executar na GPU, o speedup cai profundamente.

Essa queda profunda é causada pela transferência PCIe: é necessário copiar os N eventos para a GPU antes de fazer a busca e a comparação. A figura abaixo mostra que, à medida que N aumenta, o tempo de transferência também aumenta (representado pela cor laranja), assim como



(a) Busca binária



(b) Busca linear

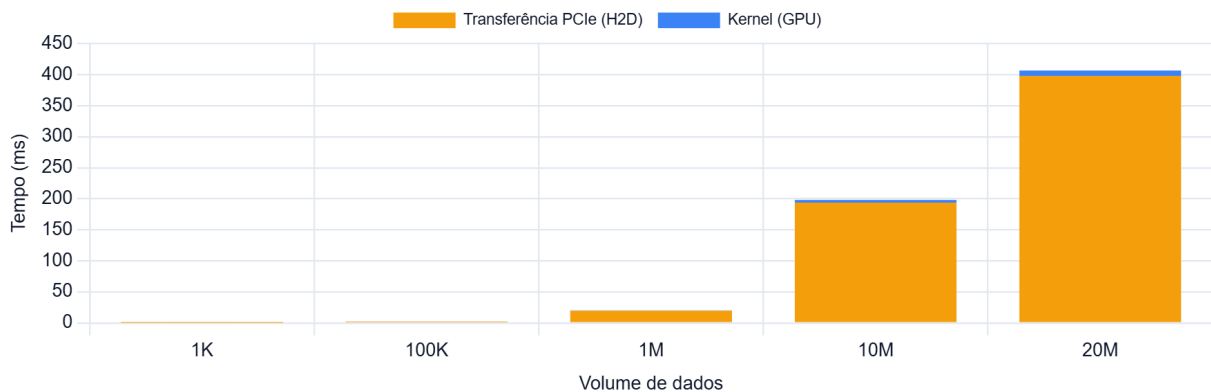


(c) Hash lookup

Figura 8 – Máquina 1. Speedup por modo de execução (serial, OpenMP com 2 a N threads e GPU) das três cargas de busca: (a) binária, (b) linear, (c) hash lookup. Linha tracejada = teto de Amdahl; linha vermelha = break-even ($S = 1$).

o tempo de execução dentro da GPU (representado pela cor azul). Observa-se que, por mais que o volume de dados transferidos cresça, a transferência representa sempre a maior fração do tempo total de execução.

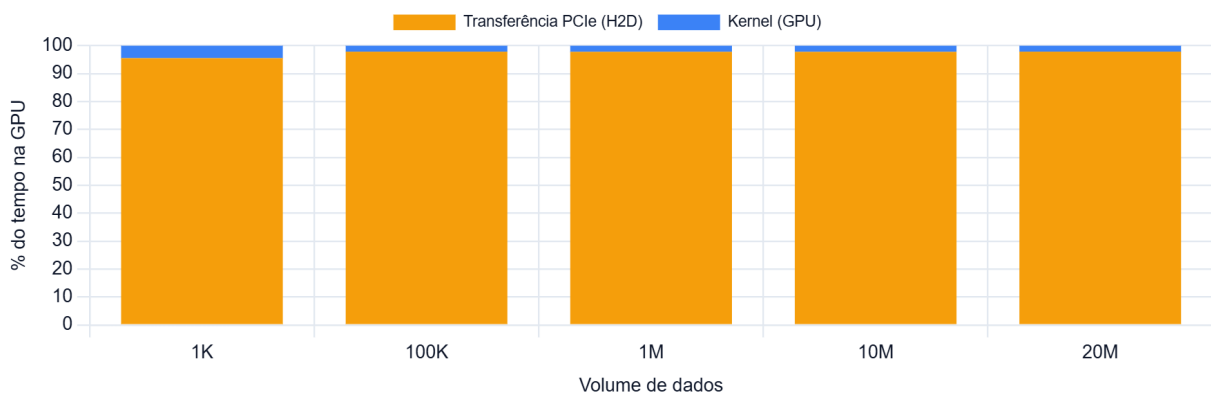
Figura 9 – Máquina 1. Decomposição do tempo total na GPU para a busca, por volume de dados (1K–20M): transferência PCIe H2D (laranja) vs. execução do kernel (azul). A transferência domina e cresce com o volume.



Fonte: Elaborada pelo autor.

A figura seguinte mostra a proporção do tempo da execução kernel na GPU e transferência PCIe exibindo que a maior parte do tempo total sempre é gasta na transferência PCIe não importa o quão grande seja o volume de dados processados a intensidade operacional e mínima.

Figura 10 – Máquina 1. Proporção do tempo na GPU entre transferência PCIe (laranja) e kernel (azul) na busca: o PCIe responde por ~98% do tempo em todos os volumes.

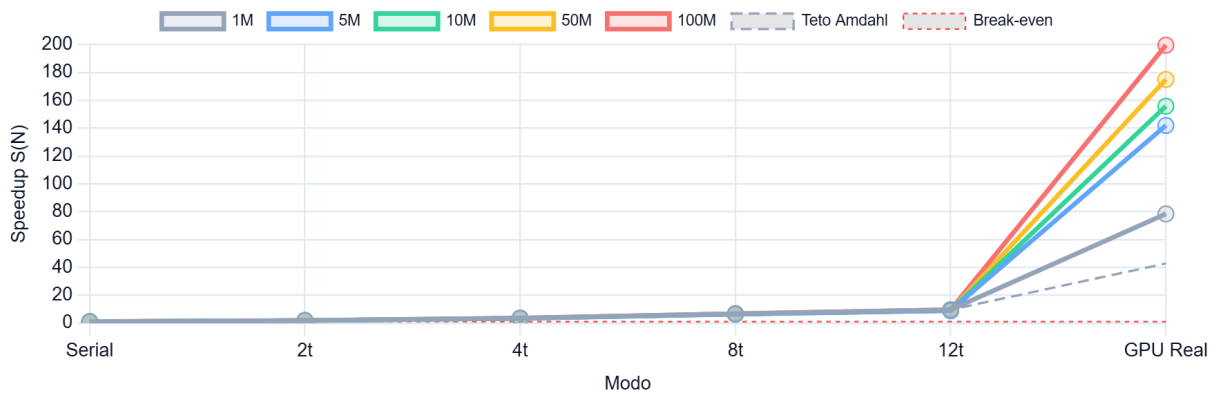


Fonte: Elaborada pelo autor.

As cargas de matemática densa, Mandelbrot e Monte Carlo, contam a história oposta. O Mandelbrot atinge fração paralela de ~98% em todas as máquinas e o Monte Carlo fica entre 84% e 96%; o speedup continua subindo com mais threads, sem sinal de saturação na faixa testada. É o comportamento descrito por Gustafson (GUSTAFSON, 1988): o problema escala com os recursos, porque o trabalho útil cresce mais rápido que a parcela serial. São cargas puramente compute-bound, em que o acesso à memória é mínimo a maior parte das variáveis é gerada dentro do próprio kernel, com pouquíssimos dados de entrada trafegando pelo barramento PCIe.

A figura abaixo mostra que, à medida que a densidade matemática ou seja mais pontos indicado por volume e a quantidade de núcleos crescem, o speedup também cresce, observando que há um salto enorme de 12t para muitas threads GPU.

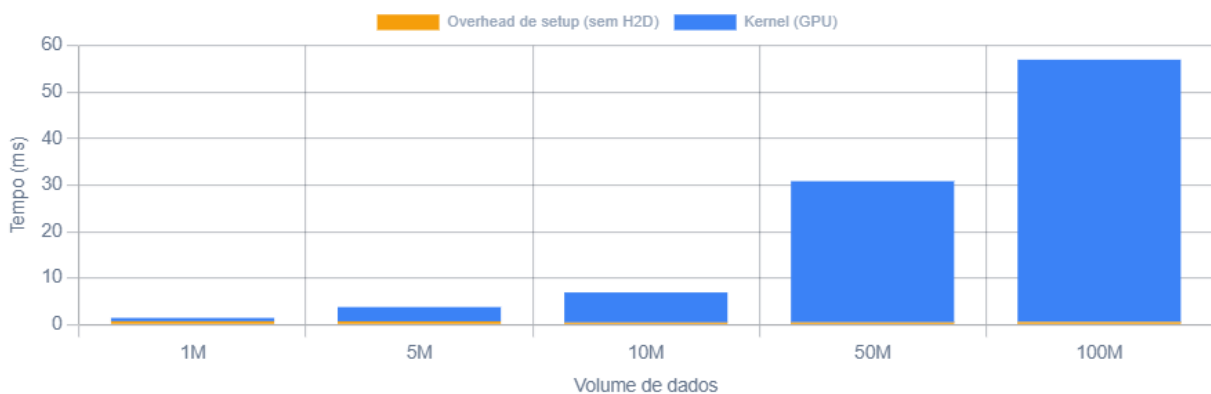
Figura 11 – Máquina 2. Speedup do Mandelbrot por modo de execução (serial → GPU), por volume (1M–100M): cresce com o volume e dispara na GPU (163–266× no maior volume dependendo da máquina), sem saturar, comportamento de Gustafson.



Fonte: Elaborada pelo autor.

A figura abaixo mostra que a maior fração do tempo é gasta na execução do kernel na GPU, e a transferência pelo barramento PCIe é mínima.

Figura 12 – Máquina 2. Decomposição do tempo do Mandelbrot na GPU, por volume: overhead de setup/transferência (laranja, mínimo) vs. kernel (azul, dominante). Oposto da busca, quase nenhum tempo em transferência PCIe.

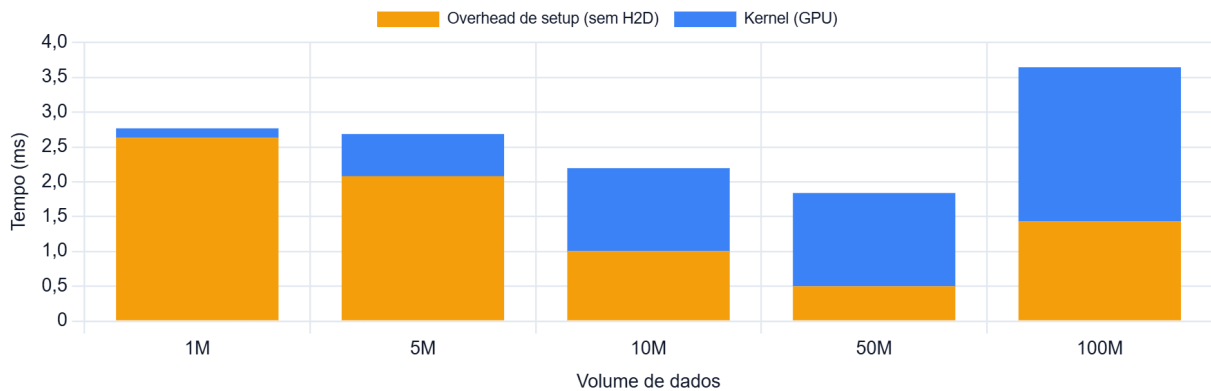


Fonte: Elaborada pelo autor.

A Máquina 3 é um caso curioso. Ela tem o paralelismo de CPU mais fraco, por limitação do número de núcleos disponíveis, mas ao mesmo tempo é a que apresenta o melhor speedup na GPU entre as máquinas do experimento um modelo superior às outras duas, com 8.704 núcleos CUDA. Esses núcleos, além de numerosos, são um pouco mais complexos que os das outras arquiteturas; fica em aberto experimentar a complexidade de núcleo com base em pesquisas como Hill & Marty. E existe uma anomalia no Monte Carlo: nessa máquina a PCIe ocupa 39% do tempo, contra 6–9% nas outras. A RTX 3080 é tão rápida que até uma carga compute-bound

começa a sentir a transferência. Isso mostra que o speedup não depende só de núcleos físicos, barramentos e cache sem falhas depende de toda a arquitetura e do dimensionamento do sistema, desde a complexidade do problema até a placa-mãe que interliga os componentes.

Figura 13 – Máquina 3. Decomposição do tempo do Monte Carlo na GPU (RTX 3080), por volume: overhead de setup/transferência PCIe (laranja) vs. kernel (azul). Diferente das outras máquinas, aqui a transferência chega a $\sim 39\%$ do tempo, a RTX 3080 calcula tão rápido que a PCIe volta a pesar mesmo numa carga compute-bound (a anomalia discutida no texto acima).



Fonte: Elaborada pelo autor.

A carga memory-bound satura na linha de teto de Amdahl porque sua fração não paralela é a própria banda de memória. Já a carga compute-bound ignora esse teto e segue subindo (Gustafson), porque quase todo o seu trabalho é paralelizável. O valor de P é decidido pela natureza da carga: acesso à memória versus aritmética.

5.2 Intensidade operacional: explicando o speedup $CPU \times GPU$

O ganho de migrar uma carga da CPU para a GPU não é constante depende da intensidade operacional (OI) da carga, quantas operações de ponto flutuante é realizado por byte que precisa trafegar pelo barramento PCIe, A Tabela 8 mostra para cada algoritmo o tempo dentro do kernel da GPU e o tempo de transferência pelo barramento PCIe e é essa separação que explica os resultados.

Um dos melhores exemplo ilustrativo é a busca binária na GPU é entre $74\times$ e $90\times$ mais lenta que a versão serial dependendo da máquina, olhando apenas o speedup no kernel na M3 (RTX 3080) o kernel atinge $4,68\times$ porém 98–100% do tempo total e realizar a copia do array de eventos da Dram para Vram pelo PCIe para uma busca que retorna poucos resultados comparado com o tamanho total, A intensidade operacional e pequena o barramento domina a banda do PCIe é menor muito menor que a da memoria da GPU.

Comparar o speedup só do kernel com o speedup total demonstra claramente o gargalo, A busca linear tem kernel entre $22\times$ e $191\times$ mais rápido dependendo da GPU, mas o total fica entre $0,47\times$ e $0,93\times$ a dependência de tráfego de memória consome todo o ganho, O Mandelbrot

Tabela 8 – Onde o tempo da GPU é gasto (kernel $\times$ PCIe), 3 máquinas

Algoritmo	Máquina	Kernel (ms)	Transfer PCIe (ms)	% em PCIe	Speedup total	Speedup só kernel
Busca binária	M1 (GTX 1650)	8,16	405,06	98%	0,01 $\times$	0,56 $\times$
Busca binária	M2 (RTX 3060)	1,90	188,18	99%	0,01 $\times$	1,35 $\times$
Busca binária	M3 (RTX 3080)	0,89	370,30	100%	0,01 $\times$	4,68 $\times$
Busca linear	M1 (GTX 1650)	8,66	398,14	98%	0,47 $\times$	21,95 $\times$
Busca linear	M2 (RTX 3060)	2,02	186,22	99%	0,93 $\times$	86,73 $\times$
Busca linear	M3 (RTX 3080)	0,95	369,37	100%	0,49 $\times$	190,81 $\times$
Hash lookup	M1 (GTX 1650)	9,91	49,70	83%	1,37 $\times$	8,26 $\times$
Hash lookup	M2 (RTX 3060)	1,38	25,85	95%	3,09 $\times$	61,03 $\times$
Hash lookup	M3 (RTX 3080)	0,62	25,84	98%	2,43 $\times$	103,96 $\times$
Monte Carlo π	M1 (GTX 1650)	14,60	1,48	9%	13,45 $\times$	14,81 $\times$
Monte Carlo π	M2 (RTX 3060)	16,63	1,13	6%	12,35 $\times$	13,19 $\times$
Monte Carlo π	M3 (RTX 3080)	2,22	1,43	39%	68,37 $\times$	112,50 $\times$
Mandelbrot	M1 (GTX 1650)	69,70	1,03	1%	163,34 $\times$	165,76 $\times$
Mandelbrot	M2 (RTX 3060)	56,42	0,57	1%	199,59 $\times$	201,61 $\times$
Mandelbrot	M3 (RTX 3080)	55,58	2,76	5%	266,27 $\times$	279,50 $\times$

Fonte: Elaborada pelo autor.

possui kernel entre 166 $\times$ e 280 $\times$ e o total entre 163 $\times$ e 266 $\times$ quase sem perda pois o tráfego de memória pelo barramento de PCIe é mínimo, o que separa os dois casos é o OI: quantos FLOPs por byte transferido.

Vale notar também que o speedup de kernel cresce da M1 para a M3 em todos os algoritmos. Como as buscas usam o mesmo volume de dados (20M) em todas as máquinas, a comparação é justa: adicionar mais núcleos CUDA melhora o speedup do kernel. O que os núcleos não resolvem é o tempo de transferência, nas cargas memory-bound, a PCIe continua limitando o tempo total.

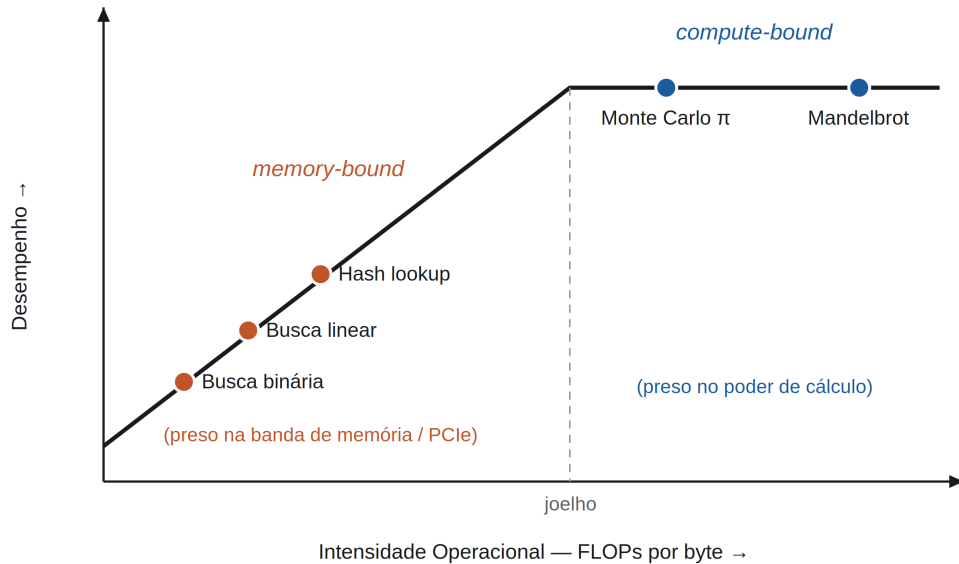
A OI é sempre a mesma divisão: **operações que o algoritmo faz $\div$ bytes que ele precisa buscar na memória**. Ambos são propriedades do *algoritmo*, não do processador. Está sendo considerado parte do calculo as operações logica, aritmética ou ponto flutuante.

Tabela 9 – Intensidade Operacional: a conta por unidade de trabalho (estimativa de ordem de grandeza a partir do código)

Algoritmo	Operações por unidade	Bytes movidos por unidade	OI $\approx (\text{ops} \div \text{bytes})$	Região
Busca binária	~ 1 comparação por acesso	96 B (a struct Event inteira)	$\sim 1 \div 96 \approx \mathbf{0,01}$	memory-bound
Busca linear	~ 2 comparações por elemento	96 B (a struct Event inteira)	$\sim 2 \div 96 \approx \mathbf{0,02}$	memory-bound
Hash lookup	~ 2 ops (hash) + 1 salto de ponteiro	$\sim 64\text{--}96$ B (acesso aleatório $\rightarrow$ cache miss)	$\sim 2 \div 80 \approx \mathbf{0,02}$	memory-bound
Monte Carlo π	~ 10 FLOPs por amostra ($2 \times \text{LCG} + x^2 + y^2 + \text{comparação}$)	≈ 0 B (x, y nascem no kernel)	$\sim 10 \div \sim 0 \rightarrow \mathbf{\text{alta}}$	compute-bound
Mandelbrot	$\sim 100\text{--}1000$ FLOPs por pixel (~ 8 FLOPs $\times$ até 256 iterações)	≈ 0 B (a iteração nasce no kernel)	$\sim \text{centenas} \div \sim 0 \rightarrow \mathbf{\text{alta}}$	compute-bound

Fonte: Elaborada pelo autor.

Figura 14 – Modelo Roofline com os cinco algoritmos. As três buscas caem na rampa (memory-bound, presas na banda de memória/PCIe); Monte Carlo e Mandelbrot encostam no teto (compute-bound).



Fonte: Elaborada pelo autor.

Uma OI baixa se encaixa na rampa; à medida que a OI aumenta, o desempenho chega a um teto. Ou seja, quando a OI é alta, o processador é melhor aproveitado, usando mais suas unidades internas de cálculo; quando a OI é baixa, o processador é pouco usado, e é a velocidade da memória que passa a ser o principal recurso.

Em resumo, o speedup entre CPU e GPU observado pode ser visto que a dependência de desempenho dos dois dispositivos aritmético ou de banda na intensidade operacional da carga, A GPU muito núcleos simples foco em paralelismo, vence quando a intensidade operacional é alta

o bastante para pagar os custos de transferência por meio do barramento PCIE, se o gerador de BOX-Muller fosse colocado para ser executado na GPU ele cairia na região de memory-bound assim que os dados cruzam o barramento.

6 CONSIDERAÇÕES FINAIS

Retomando a pergunta da Seção 1.1, as Leis de Amdahl e Gustafson conseguem explicar, na prática, o speedup de algoritmos distintos executados em CPU serial, OpenMP e GPU? A resposta é **sim, com ressalvas**, e a ressalva é a própria natureza da carga. As cargas de **busca obedeceram a Amdahl**: o ganho satura porque a fração que não paraleliza não é apenas código sequencial, e sim, em grande parte, o tráfego de memória. A busca linear confirmou o ganho com o aumento do volume, mas presa ao teto memory-bound (H1 confirmada, com a ressalva do overhead em volume pequeno), e o hash escalou menos que o esperado não apenas pelas colisões, e sim pelo acesso aleatório à memória (H2 parcial). As cargas de **matemática densa obedeceram a Gustafson**: o speedup escala com os recursos e não satura na faixa testada e, mesmo considerando apenas o tempo de kernel, o ganho continua crescendo com os recursos, ainda que não de forma perfeitamente linear (H3 confirmada). A previsão de que a busca linear venceria na GPU foi **refutada** não pelo cálculo, mas pela transferência PCIe, que se comportou como uma **fração serial física**: domina o tempo quando a intensidade operacional é baixa e desaparece quando é alta. Ou seja, as leis clássicas continuam válidas, desde que se leia a “fração serial” não como linhas de código, mas como o recurso que a carga realmente disputa memória ou aritmética.

Na prática de engenharia, isso significa que algoritmos com o mesmo objetivo como as três buscas se comportam de forma distinta por causa de suas instruções internas: a maior parte do ganho de desempenho está em ter paralelismo disponível e em saber o momento certo de usá-lo. Conhecer o percentual de fração serial e paralela de uma carga é o que permite decidir quando vale adicionar mais núcleos e de qual tipo muitos núcleos simples ou poucos núcleos complexos. Quando a aplicação tem baixa intensidade operacional, esse ganho passa a depender das propriedades da memória, e não do poder de cálculo.

As Leis de Amdahl e Gustafson são um ponto de partida sólido para entender o paralelismo, mas, à medida que se faz a experimentação científica, algumas brechas vão se abrindo e outros modelos clássicos as preenchem: o Roofline mostra onde mora o gargalo, e Hill & Marty (2008), mesmo sendo de 2008, anteciparam uma visão que hoje se materializa nos chips assimétricos e heterogêneos, como o Apple Silicon e o NVIDIA Spark. Por fim, o volume de dados de fato influencia o comportamento do speedup: nas buscas os dados são passados massivamente no maior volume testado, 20.000.000 de eventos ocupam cerca de 1,8 GB brutos enquanto na matemática densa entram apenas como poucos argumentos, já que todo valor necessário é calculado e gerado dentro do próprio kernel.

6.1 Trabalhos futuros: o overhead externo da computação heterogênea multi-processador

Este trabalho abordou extensamente a limitação imposta pelo barramento PCIe, e disso surge uma pergunta natural: como esse gargalo se comporta em cenários com múltiplas GPUs? A abordagem é enxergar o paralelismo em outra escala, cada GPU passa a representar um único processador, e a divisão de trabalho volta a esbarrar no barramento, só que agora não uma vez, e sim uma vez por dispositivo. O ponto interessante está na transferência: se copiar um volume de dados V da DRAM para a VRAM leva um tempo T de, por exemplo, 200 ms, quebrar esse volume em quatro partes para quatro GPUs faria cada fatia mover $V/4$, e, no caso ideal, o tempo de transferência cairia para cerca de 50 ms. Esse ganho, porém, só se concretiza se as quatro transferências forem de fato paralelas: cada GPU precisa de uma via PCIe independente e a banda de memória do host precisa sustentar as quatro cópias simultâneas. Se o *root complex* PCIe ou a memória do host forem um gargalo compartilhado, as transferências disputam a mesma banda e o tempo real fica acima dos 50 ms. Somado a isso, a mesclagem final dos resultados e a comunicação entre os dispositivos passam a compor uma nova fração serial, a mesma lógica de Amdahl, agora um nível acima. Ou seja, dividir os dados entre GPUs pode reduzir o custo de transferência de cada uma, mas não é um ganho linear garantido, e fica em aberto medir em qual classe de carga esse balanço compensa.

A anomalia observada na Máquina 3 é uma pista concreta nessa direção: uma GPU rápida o suficiente (RTX 3080) fez a transferência PCIe voltar a pesar 39% do tempo até em uma carga compute-bound, quando o cálculo deixa de ser o gargalo, é a movimentação de dados que reaparece como limite. Esse comportamento dialoga com Hill & Marty (2008), ao tratar o paralelismo sob restrição de recursos; com o fenômeno conhecido como *memory wall*, a defasagem crescente entre a velocidade de processamento e a de acesso à memória; e com os modelos de comunicação inter-processo típicos de MPI, em que a troca de dados entre nós é o análogo externo da cópia H2D. Uma continuação natural deste trabalho seria medir a mesma carga em dois ou mais dispositivos, separando o tempo de particionamento e de comunicação do tempo de kernel, para verificar se, e em qual classe de carga, a fração serial de fato cresce com o número de dispositivos.

REFERÊNCIAS

Accelerated Computing Academy. **Roofline**. 2025. Disponível em: <<https://accelerated-computing.academy/fall25/resources/roofline/>>. Acesso em: 29 de jun. de 2026.

AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: **AFIPS Spring Joint Computer Conference**. [s.n.], 1967. Disponível em: <<https://www3.cs.stonybrook.edu/~rezaul/Spring-2012/CSE613/reading/Amdahl-1967.pdf>>. Acesso em: 4 de jul. de 2026.

BOX, G. E. P.; MULLER, M. E. A note on the generation of random nor-

mal deviates. **The Annals of Mathematical Statistics**, 1958. Disponível em: <<https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-29/issue-2/A-Note-on-the-Generation-of-Random-Normal-Deviates/10.1214/aoms/1177706645.full>>. Acesso em: 4 de jul. de 2026.

GUSTAFSON, J. L. Reevaluating amdahl's law. **Communications of the ACM**, 1988. Disponível em: <<https://dl.acm.org/doi/pdf/10.1145/42411.42415>>. Acesso em: 4 de jul. de 2026.

HILL, M. D.; MARTY, M. R. Amdahl's law in the multicore era. **IEEE Computer**, 2008. Disponível em: <https://research.cs.wisc.edu/multifacet/papers/tr1593_amdahl_multicore.pdf>. Acesso em: 4 de jul. de 2026.

MOORE, G. E. Cramming more components onto integrated circuits. **Electronics**, 1965. Disponível em: <<https://www.cs.utexas.edu/~fussell/courses/cs352h/papers/moore.pdf>>. Acesso em: 4 de jul. de 2026.

[PEREIRA, M. P. A. **parallel-laws-benchmark: código-fonte do benchmark (C, OpenMP e CUDA)**. 2026. Disponível em: <<https://github.com/PmarcosEng/parallel-laws-benchmark>>. Acesso em: 4 de jul. de 2026.

[PEREIRA, M. P. A.; FAJARDO, V. A. **Benchmark das Leis de Amdahl e Gustafson: projeto (etapa 1)**. 2026. Disponível em: <https://github.com/PmarcosEng/parallel-laws-benchmark/blob/main/docs_projeto/Artigo_Projeto.pdf>. Acesso em: 4 de jul. de 2026.

POLLACK, F. J. New microarchitecture challenges in the coming generations of cmos process technologies. In: **Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture (MICRO-32)**. [s.n.], 1999. Disponível em: <<https://research.ac.upc.edu/HPCseminar/SEM9900/Pollack1.pdf>>. Acesso em: 4 de jul. de 2026.

WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: An insightful visual performance model for multicore architectures. **Communications of the ACM**, 2009. Disponível em: <<https://people.eecs.berkeley.edu/~kubitron/cs252/handouts/papers/RooflineVyNoYellow.pdf>>. Acesso em: 4 de jul. de 2026.

APÊNDICE A – Repositório do código-fonte

Repositório: <<https://github.com/PmarcosEng/parallel-laws-benchmark>>

Código-fonte completo do benchmark: todos os algoritmos em C (serial), OpenMP e CUDA, os scripts de medição e os resultados por máquina.